# Baseline Retrieval vs. Cross-Encoder Re-Ranking for RAG on a Legal Corpus

Anurag Kaushal
Roll No. 2025202013, Course: LMA, RAG Assignment Report
Code and results: github.com/Anurag-1205/rag_assign1

**Abstract.** Retrieval-augmented generation can only answer correctly if the passage that holds the answer reaches the language model. I compare two retrieval pipelines over the same 230 chunks of India's Digital Personal Data Protection Act, 2023 and Rules, 2025. The baseline passes the top 3 of 10 bi-encoder candidates to Gemini; the second pipeline re-ranks the same 10 candidates with a cross-encoder before taking the top 3. On 112 stress-test queries (distractor, exception and negation), re-ranking raised Hit@3 from 0.518 to 0.589 and MRR@10 from 0.473 to 0.533. It moved the gold chunk into the LLM's context for 11 queries and out of it for 3, and in all six rescued cases that I inspected end to end the answer went from wrong or incomplete to correct. The price is latency: median retrieval time on CPU rose from 12.6 ms to 374.4 ms. For 34 queries the gold chunk was never among the 10 candidates, so first-stage recall remains the main limit.

*Keywords:* retrieval-augmented generation, dense retrieval, cross-encoder re-ranking, legal documents

## 1. Introduction and Dataset

In this assignment I compare a standard dense-retrieval baseline against a two-stage pipeline that adds cross-encoder re-ranking on top of the same retrieved candidates. My aim is to examine not only whether re-ranking improves answer quality on adversarial queries, but also to understand where it fails and what the latency trade-off looks like in practice.

I chose the Digital Personal Data Protection Act, 2023 [1] together with the Digital Personal Data Protection Rules, 2025 [2] as the corpus. These two documents were merged into a single 39-page PDF. Legal text is a practical stress-test for retrieval systems: the answer to a question is often a short qualification hidden beside a broader rule (an exception, a prohibition, or a role-specific amount), while large portions of surrounding text discuss very similar topics using overlapping vocabulary.

After cleaning the raw PDF, I split the text into 230 chunks of 120 words each with a 30-word overlap between adjacent chunks. The overlap ensures that a key clause sitting on a chunk boundary is not split across two retrieval units. All experiments below use this fixed set of 230 chunks; nothing about the corpus changes between pipelines.

## 2. Architecture Comparison

The two pipelines share an identical corpus, retrieval budget (top-10 candidates per query), and LLM call. The only difference is what happens between the initial retrieval and the LLM prompt (Table 1).

### *2.1 Bi-Encoder (Pipeline 1, Baseline)*

A bi-encoder processes the query and each document chunk independently. Both are mapped to fixed-length vectors, and similarity is measured as the cosine between them. Because chunk representations are pre-computed and stored, inference requires only one forward pass to embed the query, followed by a fast nearest-neighbour search. I used all-MiniLM-L6-v2 [3, 4] with L2-normalised embeddings and a FAISS [5] IndexFlatIP index (inner product on normalised vectors equals cosine similarity).

The weakness of a bi-encoder is that it cannot compare the query and a candidate directly. It reliably identifies passages on the same topic, but can miss the one word that changes the legal meaning. For instance, two chunks describing different members' salaries will have nearly identical embeddings, even though only one answers the specific query.

### *2.2 Cross-Encoder (Pipeline 2, Re-Ranking)*

A cross-encoder concatenates the query and a candidate chunk into a single input and runs a joint self-attention pass over both. This lets it compare wording directly and attend to phrases such as 'not entitled', 'except', and specific role names simultaneously. The relevance score is therefore much better calibrated to the actual question. I used ms-marco-MiniLM-L-6-v2, a lightweight model fine-tuned on passage re-ranking. Because the

cross-encoder cannot reuse stored document vectors, I apply it only to the 10 candidates the bi-encoder already selected, keeping the latency increase bounded.

**Table 1.** System configuration for both pipelines.

| Component | Pipeline 1 (Baseline) | Pipeline 2 (Re-Ranked) |
|---|---|---|
| First-stage retriever | all-MiniLM-L6-v2 bi-encoder, L2-normalised embeddings (same in both) | |
| Vector index | FAISS IndexFlatIP, exact inner-product search (= cosine) | |
| Candidates retrieved | Top 10 chunks per query (the same 10 in both) | |
| Re-ranking model | None | ms-marco-MiniLM-L-6-v2 cross-encoder |
| Context to LLM | Top 3 by bi-encoder score | Top 3 by cross-encoder score |
| Answer model | gemini-3.7-flash, temperature 0, identical prompt | |
| Evaluation queries | 112 stress-test queries (28 distractor, 73 exception, 11 negation), each with its gold chunk(s), i.e. the chunk(s) holding the answer | |

## 3. Results: Retrieval Quality and Answer Impact

Table 2 summarises the gold-chunk retrieval performance across all 112 queries. I focus on Hit@3 because the LLM sees only three chunks; getting the gold evidence into that context window is the necessary condition for a correct answer. Re-ranking improved Hit@3 by 7.1 percentage points and MRR@10 by 6.0 points. Hit@10 is unchanged because the cross-encoder only reorders the initial ten candidates and cannot recover gold chunks that the bi-encoder missed entirely.

**Table 2.** Gold-chunk retrieval performance across 112 stress-test queries.

| Metric | Baseline (P1) | Re-Ranked (P2) | Change |
|---|---|---|---|
| Hit@1 | 0.393 | 0.446 | +0.053 |
| Hit@3 | 0.518 | 0.589 | +0.071 |
| Hit@10 | 0.696 | 0.696 | 0.000 |
| MRR@10 | 0.473 | 0.533 | +0.060 |

Table 3 breaks down what happened to the gold chunk for each query after re-ranking. The most important column is ‘Fixed’: for these 11 queries re-ranking moved the gold chunk into the three chunks the LLM sees. The 34 queries in ‘Not in top 10’ (30 of them exception queries) expose the ceiling of the two-stage design. No amount of re-ranking can help if the first-stage retriever did not include the gold chunk in its candidate list.

**Table 3.** What happened to the gold chunk after re-ranking, by query type.

| Query type | Already in top 3 | Fixed (moved into top 3) | Still outside top 3 | Dropped out of top 3 | Not in top 10 | Total |
|---|---|---|---|---|---|---|
| Distractor | 20 | 5 | 1 | 0 | 2 | 28 |
| Exception | 30 | 4 | 7 | 2 | 30 | 73 |
| Negation | 5 | 2 | 1 | 1 | 2 | 11 |
| **All** | **55** | **11** | **9** | **3** | **34** | **112** |

### *3.1 Representative Rank Shifts*

Table 4 shows seven queries selected for deep inspection: six where re-ranking rescued the gold chunk, and one (EXC-20) where it made things worse; Figure 1 plots the same rank shifts. The clearest case is DIS-05, where a single different chunk in the context (the gold chunk moved from rank 6 to rank 2) caused the answer to change from an incorrect salary (Rs 4,00,000, the figure for the other Members) to the correct one (Rs 4,50,000).

**Table 4.** Rank of the gold chunk before and after re-ranking, with observed answer impact (DIS = distractor, NEG = negation, EXC = exception).

| Case | Query (abbreviated) | Rank shift | Answer 1 | Answer 2 |
|---|---|---|---|---|
| DIS-15 | Penalty for breaking a voluntary undertaking? | 7 → 1 | Omitted | Correct |
| DIS-05 | Monthly salary of the Chairperson? | 6 → 2 | Wrong (Rs 4,00,000) | Correct |
| NEG-10 | Are Board Members paid a sitting fee? | 5 → 1 | Omitted | Correct |
| NEG-06 | Can a Consent Manager read the data it routes? | 5 → 3 | Omitted | Correct |
| DIS-18 | Who selects the Chairperson? | 9 → 3 | Omitted | Correct |
| EXC-63 | Are courts exempt from the Act's duties? | 4 → 2 | Omitted | Correct |
| EXC-20 | Indian firm processing foreign customers' data? | 1 → 7 | Correct | Omitted |

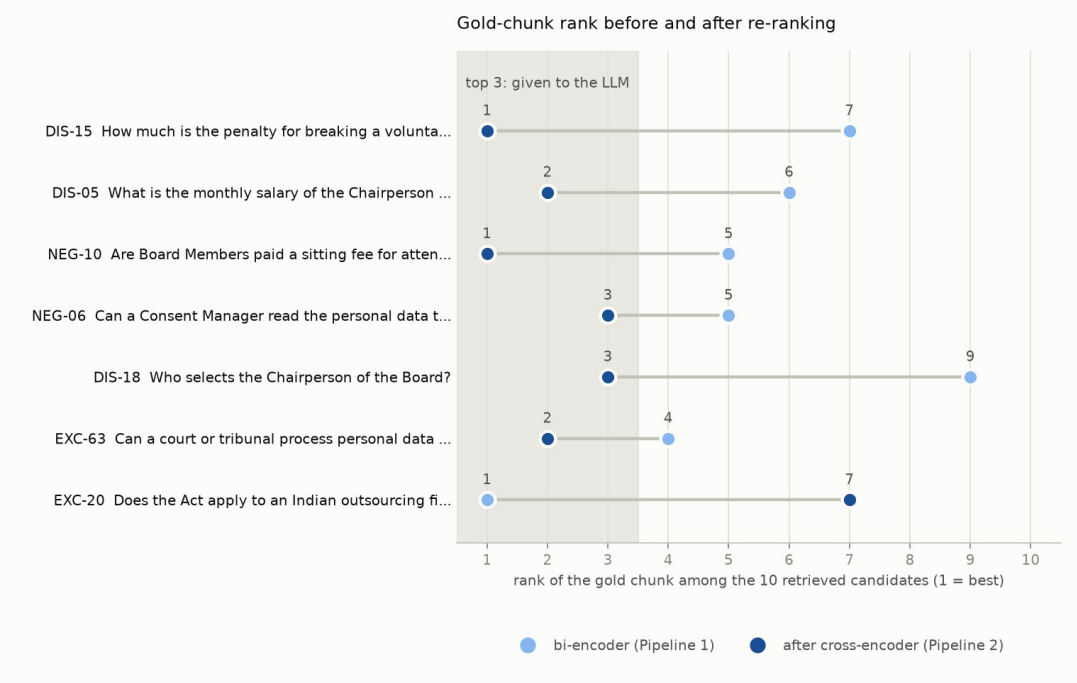



**Figure 1.** Rank of the gold chunk before and after re-ranking. Ranks 1 to 3 (shaded) are the slots visible to the LLM.

## 4. Latency Cost

Latency was measured on CPU (AMD Ryzen 5 7520U) over 180 timed runs (six rescued queries, each repeated 30 times after a warm-up pass). The Gemini API call is network-bound and excluded from the retrieval figures (Table 5, Figure 2); it is reported as a pooled median separately.

**Table 5.** Retrieval latency per query on CPU, 180 timed runs (Gemini call excluded).

| Stage | Median (ms) | p95 (ms) |
|---|---|---|
| Query embedding (bi-encoder) | 12.50 | 14.26 |
| FAISS search (top 10) | 0.07 | 0.12 |
| **Pipeline 1 retrieval total** | **12.58** | **14.40** |
| Cross-encoder scoring (10 pairs) | 361.46 | 384.00 |
| **Pipeline 2 retrieval total** | **374.40** | **395.89** |

The cross-encoder accounts for almost all of the extra 361.8 ms, roughly 30 times the cost of the baseline retrieval stage. In this setup the Gemini call dominates end-to-end time at roughly 7.24 seconds (pooled median of the 14 saved calls), so the retrieval difference adds only about 5% to the total (7.25 s vs. 7.61 s end to end). In a high-throughput system or with a faster local answer model, the retrieval cost would become the bottleneck.

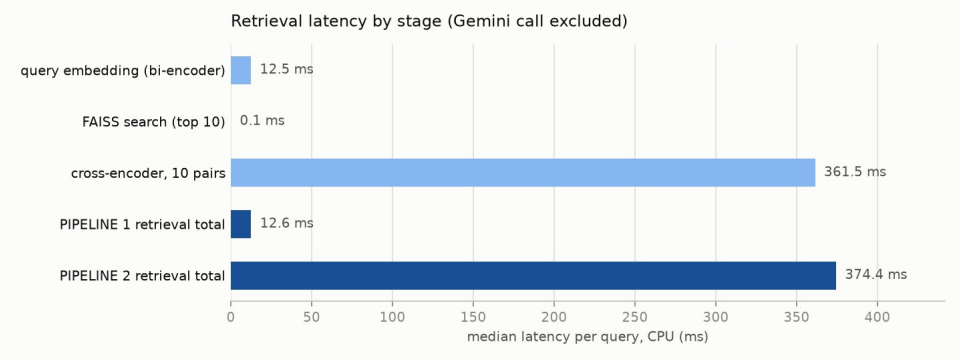



**Figure 2.** Median retrieval latency by stage. Gemini generation time is excluded.

## 5. Scaling and Conclusion

Because a cross-encoder must run a fresh forward pass for every (query, chunk) pair, exhaustive re-ranking does not scale. On this CPU, scoring all 230 chunks took 9.75 seconds (roughly 42.4 ms per pair). At that rate, a single query would need about 7 minutes for 10,000 chunks, 71 minutes for 100,000 chunks and almost 12 hours for one million chunks.

By contrast, a FAISS exact search over 10,000 vectors takes 1.78 ms and over 100,000 vectors just 17.57 ms. This is why the two-stage design is the standard: the bi-encoder provides fast, scalable recall over the full collection, and the cross-encoder spends its compute budget only on a short candidate list. The remaining weakness is clear: 34 of the 112 queries had their gold chunk outside the initial top 10, so re-ranking had no opportunity to recover them.

Cross-encoder re-ranking improved Hit@3 by 7.1 percentage points and corrected concrete answer failures including a wrong salary figure, missing prohibitions, and an overlooked legal exception. The cost was real: median retrieval latency rose from 12.58 ms to 374.40 ms, and three queries were made worse. Re-ranking is best understood as a precision improvement within a fixed recall budget, not a recall improvement. Improving first-stage recall, for instance through hybrid sparse-dense retrieval or a larger candidate pool, is likely to have a greater long-term impact than refining the re-ranking model alone.

## References

[1] *The Digital Personal Data Protection Act, 2023* (No. 22 of 2023), Gazette of India, Extraordinary, Ministry of Law and Justice, 11 Aug. 2023.

[2] *The Digital Personal Data Protection Rules, 2025*, G.S.R. 846(E), Gazette of India, Extraordinary, Ministry of Electronics and Information Technology, 13 Nov. 2025.

[3] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982-3992.

[4] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Proc. NeurIPS*, 2020.

[5] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535-547, 2021.